

Prueba Técnica para Desarrolladores: SPRING (JAVA) y Front-end (Angular) Makers

Instrucciones para el Candidato

- La prueba se divide en dos partes: teórica (30%) y práctica (70%).
- Se espera que entregues el código de la parte práctica en un **repositorio público de GitHub**.
- Explica y justifica tus decisiones técnicas en ambas partes.
- **Duración estimada:** 2 horas para la parte práctica, 60 minutos para la teórica.

Parte Teórica (30%)

1. **Optimización en sistemas financieros:**
 - a. Imagina que trabajas en una aplicación bancaria que debe procesar transacciones en tiempo real. ¿Qué estrategias aplicarías para mejorar el rendimiento y la escalabilidad? Considera aspectos como concurrencia, caché y patrones de diseño.
2. **Seguridad en APIs financieras:**
 - a. Explica cómo protegerías una API que maneja información sensible de cuentas bancarias contra ataques como inyección SQL, CSRF, XSS y otros ataques comunes.
3. **Transacciones en sistemas distribuidos:**
 - a. En un sistema bancario distribuido, ¿cómo implementarías la consistencia y el manejo de errores en una API que procesa transferencias entre cuentas en diferentes servicios?
4. **Pruebas unitarias y de integración:**
 - a. Describe cómo diseñarías una suite de pruebas que asegure la correcta operación de una API bancaria, considerando tanto pruebas unitarias como de integración. ¿Qué herramientas utilizarías?
5. **Front-end:**
 - a. En una aplicación bancaria que muestra el saldo de las cuentas, ¿cómo gestionarías el estado y la autenticación en el front-end, garantizando la seguridad y la coherencia de los datos?
6. **SPRING BOOT**
 - a. Que pasa en una aplicación internamente cuando usas la anotación **@SpringBootApplication** y cómo afecta el arranque de una aplicación?
 - b. ¿Cómo funciona el ciclo de vida de un **beans** en Spring y como podrías intervenir en él?
 - c. ¿Cómo personalizarías el comportamiento de la auto-configuración en Spring Boot sin romper la filosofía de 'convención sobre configuración'?

Parte Práctica (70%)

Desafío Práctico: Simulación de un Sistema de Gestión de Préstamos Bancarios

Debes desarrollar una **API Rest con Spring Boot** para gestionar un **sistema de préstamos bancarios**. El sistema debe permitir a los usuarios:

1. **Solicitar un préstamo:** Los usuarios pueden solicitar un préstamo indicando el monto y el plazo.
2. **Aprobar o rechazar préstamos:** Los administradores del sistema pueden aprobar o rechazar las solicitudes de préstamo.
3. **Ver estado de los préstamos:** Los usuarios pueden consultar el estado de sus préstamos aprobados o rechazados.

Además, implementa una interfaz front-end en **Angular** o **Thymeleaf** que consuma esta API. La interfaz debe permitir a los usuarios:

- Enviar solicitudes de préstamo.
- Consultar el estado de sus préstamos.
- Los administradores pueden ver y cambiar el estado de los préstamos.

Requerimientos:

- **Backend (Spring boot API REST, Spring Data(JPA), Spring Security):**
 - Implementar el sistema CRUD para préstamos y usuarios, respetando las reglas de negocio deberá usar **Spring Data JPA**, Hibernate Validator, use **Spring WebFlux** en los casos que crea que lo amerita.
 - Aplicar un patrón de **arquitectura limpia** o **hexagonal** para separar las capas de negocio, infraestructura y aplicación.
 - Implementar **caché** para reducir las consultas repetitivas sobre el estado de los préstamos. Se recomienda usar EHCACHE o cualquier otro sistema de cache de su conocimiento.
 - Asegurar la API usando **Spring Security**, asegurando que solo los administradores puedan aprobar o rechazar préstamos.
 - Implementar **transacciones** en el manejo de aprobaciones de préstamos, asegurando que las acciones sean consistentes.
 - Incluir **test unitarios** para las operaciones críticas de la API (Spring Boot starter test, JUnit 5).
 - Manejar adecuadamente los errores, devolviendo respuestas HTTP claras.
 - Se recomienda utilizar cualquier bases de datos relacionales, como PostgreSQL o MySQL (en caso de tenerlo configurado) o motores de base de datos en memoria como H2.
- **Front-end (Angular):**

- Crear una interfaz amigable que permita a los usuarios enviar solicitudes de préstamo y consultar su estado.
- Implementar la autenticación y autorización mediante **JWT**, asegurando que solo los administradores puedan aprobar o rechazar préstamos utilizando **Guards**.
- Gestionar el estado de la aplicación de manera segura (usar servicios/NgRx).
- Aplicar validaciones en el front-end para asegurar la correcta entrada de datos usando **Reactive Forms**.
- **Si la prueba es solo Dev frontend puedes usar una Api publica existente, usar mocks de datos o local storage y debes hacer uso de estilos de un nivel más avanzado que mejore la experiencia de usuario.**

- La siguiente es la posible interfaz que ejemplifica el requerimiento buscado:

Usuario: email: [usuario@test.com], password: 123

Admin: email: [admin@test.com] password: 123

1. Login

Iniciar Sesión

Ingrese sus credenciales para acceder

Email

Contraseña

Ingresar

2. Usuario que solicita préstamo:

Bienvenido, Usuario [Cerrar Sesión](#)

Préstamos

Solicitar Préstamo

Ingrese el monto que desea solicitar

Monto

Solicitar

Mis Préstamos

Monto: \$1000 - Estado: pendiente

Monto: \$2000 - Estado: aprobado

Monto: \$30000 - Estado: pendiente

3. Administrados que aprueba préstamo

Hola, Admin Admin

[Cerrar Sesión](#)

Gestionar Solicitudes de Préstamos

Usuario: usuario@test.com - Monto: \$1000 - Estado: pendiente

Aprobar

Rechazar

Usuario: usuario@test.com - Monto: \$2000 - Estado: aprobado

*puedes hacer todo con datos fake para adelantar la interfaz sin depender del backend.